

EVE Healthcare

SDE Intern — Backend Engineering Assignment

Estimated time: 3–4 hours | **Primary focus:** Backend engineering, APIs, databases & system design

Assignment Overview

Build a small backend service for diagnostic test bookings and simulated payments. The goal is to evaluate your ability to design clean APIs, model data correctly, handle real-world edge cases, and write maintainable code.

You may use Python/Django, FastAPI, Flask, or another backend framework you are comfortable with. PostgreSQL is preferred.

1. Authentication

Implement basic user authentication:

- User signup
- User login
- JWT-based authentication
- Basic request validation

2. Diagnostic Centres & Tests

Create APIs to manage and retrieve diagnostic centres and the tests they offer. Each centre should have at least:

- Centre name
- Location
- Available diagnostic tests
- Test price

3. Booking System

Authenticated users should be able to book a diagnostic test. A booking should contain:

- Patient/user
- Diagnostic test
- Diagnostic centre
- Appointment date/time
- Amount
- Booking status

Suggested booking states: **PENDING**, **CONFIRMED**, **FAILED**, and **CANCELLED**.

4. Simulated Payment Service

Do not integrate a real payment gateway. Create a mock payment endpoint that simulates payment processing.

`POST /payments/`

The simulated payment should result in either **SUCCESS** or **FAILED**. Update the related booking accordingly.

5. Payment Webhook

Create a webhook endpoint that accepts payment-status updates from a simulated payment provider.

```
POST /payments/webhook/
```

Important: The webhook must be idempotent. If the same webhook/event is received multiple times, it must not create duplicate payments, duplicate bookings, or otherwise corrupt the booking state.

6. Edge Cases

Your implementation should thoughtfully handle cases such as invalid requests, repeated webhook events, invalid booking IDs, failed payments, and attempts to modify resources without authorization.

Optional Bonus

These are not required, but can help demonstrate additional engineering ability:

- Redis caching
- Celery/background jobs
- Docker & docker-compose
- Swagger/OpenAPI documentation
- Unit/integration tests
- Structured logging
- Pagination
- Rate limiting
- Retry handling for webhook processing

Submission Requirements

Submit a GitHub repository containing:

```
README.md
requirements.txt or pyproject.toml
Dockerfile (if Docker is used)
docker-compose.yml (if applicable)
Source code
Tests
```

Your README should clearly explain:

- How to run the project locally
- API endpoints and example requests
- Database/schema design
- Important assumptions you made
- What you would improve if you had more time

Evaluation Criteria

Area	Weight
Code quality & maintainability	20%
API/backend design	20%
Database design	15%

Area	Weight
Edge-case handling	15%
Tests	10%
Git/README/documentation	10%
Bonus engineering	10%

What We Look For

We care more about your engineering thinking than the number of features you implement. A smaller, well-designed, tested solution is better than a large codebase with poor structure.

During the next interview stage, you may be asked to explain your architecture and make a small change to your implementation live.

Good luck — and have fun building!

Team EVE Healthcare